

Course Notes – Applied Mathematics and Computing

Part I	Methodological Aspects of Machine Learning
Part II	Prediction in Temporal Series

PART I — Methodological Aspects of Machine Learning

Machine learning models do not exist in a vacuum. Before a model can be trained and before its results can be trusted, a rigorous methodological pipeline must be followed. This part covers three fundamental methodological pillars: how to select and transform the input data, how to evaluate the quality of a learned hypothesis, and how to statistically compare two competing hypotheses using the paired t-test.

1. Data Selection and Transformation

1.1 Selection of Input Data

The way training examples are chosen has a direct impact on what a model can learn. Several strategies are commonly used:

- **Random selection:** examples are drawn uniformly at random from the available pool.
- **Most similar / most different:** examples are chosen to maximise diversity or, conversely, to focus on a cluster of interest.
- **Following a distribution:** sampling respects a known or estimated underlying distribution.
- **Class-border instances:** examples near decision boundaries are prioritised because they carry the most discriminative information.
- **Boosting:** each example is assigned a weight proportional to its current classification error. Misclassified examples receive higher weight in subsequent rounds, forcing the learner to focus on hard cases.
- **Incremental learning:** new data is incorporated continuously, without retraining the model from scratch.

1.2 Pre-processing

Raw data rarely arrives in a form suitable for direct use. Two pre-processing steps are especially important:

- **Noise reduction:** corrupted or inconsistent measurements are smoothed or filtered before training.
- **Unknown / missing data treatment:** missing values must be handled — by imputation, removal, or encoding — rather than ignored.

1.3 Feature Selection

Feature selection is a form of dimensionality reduction whose goal is to retain only the attributes that are most informative for the target task. Three broad families of techniques exist:

- **Feature ranking / filtering:** each feature is scored individually using a criterion such as Pearson correlation, entropy, or mutual information; features below a threshold are discarded.

- **Subset selection (wrappers):** candidate feature subsets are evaluated by actually training a model on them and measuring predictive performance. The search navigates the lattice of all possible subsets using hill-climbing, best-first search, or other strategies.
- **Embedded methods:** feature selection is integrated into the model learning step itself (e.g., regularised regression).

The search process for subset selection requires three components:

- **Search algorithm:** any algorithm that can traverse the feature-subset space (e.g., greedy forward/backward selection).
- **Evaluation criterion:** correlation, entropy, mutual information, or direct error on a held-out set.
- **Stopping criterion:** a fixed number of iterations, a percentage threshold, or an absolute performance value.

Wrapper architecture

In a wrapper, the feature-subset search module iterates over candidate subsets A' . For each candidate, the inductive learning algorithm is called on a training set augmented with only those features, and the resulting error feeds back into the search. The final output is the subset A'' that yielded the best performance. Because wrappers couple feature selection tightly to the learner, they tend to produce better feature sets at the cost of higher computational expense.

1.4 Principal Component Analysis (PCA)

PCA is a classical dimensionality-reduction technique applicable to numeric attributes. It projects the original n -dimensional feature space onto a reduced p -dimensional space ($p \leq n$):

The new components f_i satisfy two key properties:

- **Mutual independence:** the components are uncorrelated with one another.
- **Ordered by relevance:** f_1 captures the most variance, f_2 the second most, and so on. This ordering allows a practitioner to select the top- k components and discard the rest.

■ **Trade-off:** PCA discards the semantic meaning of the original features. The resulting components are linear combinations of the originals and cannot be interpreted in domain terms.

1.5 Evaluation Criteria for Feature Selection

Two popular filter criteria measure the linear or statistical dependence between a feature x and the target y :

Pearson Correlation Coefficient

where n is the number of examples, μ_i is the mean of feature i , and σ_i is its standard deviation. Values close to ± 1 indicate strong linear association; values near 0 indicate independence.

Mutual Information

Mutual information captures non-linear dependencies as well. In the discrete case:

In the continuous case the sums become integrals:

1.6 Feature Generation

Instead of only selecting from existing features, one can dynamically create new ones. Useful strategies include:

- Comparing attributes of the same type to yield a difference or ratio feature.
- Discretising continuous features into intervals.
- Clustering several nominal values into a coarser set.
- Counting the number of Boolean features that are true (or false) in a given context.
- Combining two attributes that co-appear in a rule.
- **Mathematical combinations** (continuous): sum, subtraction, multiplication, average.
- **Logical combinations** (discrete): conjunction, disjunction, negation.

1.7 Additional Pre-processing Operations

- **Discretisation:** applying unsupervised learning (e.g., k-means) over one or several continuous features to convert them into categorical ones.
- **Numerisation (one-hot encoding):** each nominal value of a categorical feature becomes a separate binary feature. A feature with k values yields k binary columns.
- **Range normalisation:** scales a value v to the interval $[0, 1]$ using the linear formula: $v' = (v - \min) / (\max - \min)$.
- **Self-organising maps:** a neural-network-based technique for non-linear dimensionality reduction.

2. Hypothesis Evaluation

2.1 Why Evaluate?

Hypothesis evaluation is the process of estimating the quality of a learned model. It serves two main purposes:

- **Deployment decision:** should we trust a model that was trained on only 40 instances?
- **Internal algorithm guidance:** when is it appropriate to prune a branch of a decision tree?

2.2 Challenges

- **Bias in estimation:** evaluating a model on the same data used to train it will give an overly optimistic error estimate — the model may have memorised the training set (overfitting).
- **Variance in estimation:** even on a held-out test set, the measured error fluctuates from set to set. Different test sets drawn from the same distribution can yield different error estimates.

2.3 Sample Error vs. Real Error

Given a hypothesis h , a true target function f , and a dataset S of n examples:

Sample (empirical) error:

where $\delta(f(x), h(x)) = 0$ when $h(x) = f(x)$ and 1 otherwise. This is simply the fraction of incorrectly classified examples in S .

Real (true) error:

This is the probability of misclassification on a future instance drawn from the same distribution D that generated the training data. It is what we actually care about but can never measure directly — we can only estimate it.

2.4 Decision Boundaries and Error Types

Consider the student-grade example from the lectures. We observe class exercise scores p_i and want to predict whether a student will pass (A) or fail ($\sim A$) the final exam. The conditional probability density functions $f(p_i|A)$ and $f(p_i|\sim A)$ overlap, creating unavoidable classification errors:

- **False Negatives:** the probability $P(p_i < 5 | A)$ — students who should pass but are predicted to fail because their exercise score falls below the decision boundary.
- **False Positives:** the probability $P(p_i \geq 5 | \sim A)$ — students who will fail but are predicted to pass.
- **Total Error:** the sum of false negatives and false positives.

The **minimum total error** is achieved by placing the decision boundary at the point where the two densities cross, which corresponds to minimising:

If the cost of false negatives differs from that of false positives, the decision boundary can be shifted accordingly: moving it left minimises false negatives (at the expense of more false positives), while moving it right does the opposite.

2.5 The Confusion Matrix

For binary classification, all possible outcomes can be arranged in a 2×2 confusion matrix:

	Actual: Positive	Actual: Negative
Predicted: Positive	True Positive (TP)	False Positive (FP)
Predicted: Negative	False Negative (FN)	True Negative (TN)

2.6 Performance Metrics

Metric	Formula	Interpretation
Accuracy	$(TP + TN) / (P + N)$	Overall fraction of correct predictions
Precision	$TP / (TP + FP)$	Of all predicted positives, how many are truly positive?
Recall (Sensitivity)	$TP / (TP + FN) = TP / P$	Of all true positives, how many were predicted positive?
Specificity	$TN / (TN + FP) = TN / N$	Of all true negatives, how many were predicted negative?
FP Rate	$FP / (FP + TN) = FP / N$	Of all true negatives, how many were incorrectly predicted positive?
F-Measure	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of precision and recall

Additional metrics include the **kappa coefficient** (which accounts for the possibility of agreement by chance) and the **Matthews Correlation Coefficient (MCC)** (a balanced measure useful for imbalanced datasets).

2.7 Confidence Intervals for the True Error

Since $\text{error}_S(h)$ is a random variable, we can reason about how far it is likely to deviate from $\text{error}_D(h)$. The sample error follows a **binomial distribution** with:

- Mean: $\text{error}_D(h)$
- Standard deviation: $\sqrt{[\text{error}_S(h)(1 - \text{error}_S(h)) / n]}$

When n is sufficiently large ($n \geq 30$ is a common rule of thumb), the binomial distribution is well approximated by a normal distribution with the same mean and standard deviation. This allows us to compute $N\%$ -confidence intervals analytically using the standard normal quantile z_N .

The binomial probability mass function is:

with expectation $E[X] = np$, variance $\text{Var}(X) = np(1 - p)$, and standard deviation $\sigma_X = \sqrt{[np(1 - p)]}$.

3. Hypothesis Contrast: The Paired t-Test

3.1 Motivation

Suppose we have two learning algorithms, L_A and L_B , and we want to decide which one generalises better. A single comparison on one train/test split is unreliable due to variance. The paired t-test provides a statistically principled comparison by repeatedly measuring the difference in error across multiple data folds.

3.2 Cross-Validation for the t-Test

The procedure for generating the test statistic is:

Step 1: Split the full dataset D_0 into K disjoint folds $T_1, T_2, \dots, T_K$, each of size at least 30.

Step 2: For each fold i ($i = 1$ to K): use T_i as the test set and $S_i = D_0 - T_i$ as the training set. Train both models: $h_A \leftarrow L_A(S_i)$ and $h_B \leftarrow L_B(S_i)$. Compute the difference: $\delta_i = \text{error}_{T_i}(h_A) - \text{error}_{T_i}(h_B)$.

Step 3: Return $\delta_{\blacksquare} = (1/K) \sum_{i=1}^K \delta_i$

3.3 The Paired t-Test Formula

The goal is to estimate $E_{S \in D_0} [\text{error}_D(L_A(S)) - \text{error}_D(L_B(S))]$, i.e., whether the differences in error are statistically significant. With confidence level $N\%$, the true difference lies in:

where:

- $t_{N, K-1}$ is the critical value from the Student t-distribution with $K - 1$ degrees of freedom at confidence level $N\%$.
- $S_{\delta_{\blacksquare}}$ is the estimated standard deviation of $\delta_{\blacksquare}$:

Decision rule: if the confidence interval for $\delta_{\blacksquare}$ does not include zero, we can conclude with $N\%$ confidence that L_A and L_B have statistically different generalisation performance.

PART II — Prediction in Temporal Series

Many real-world datasets arrive as sequences of observations ordered in time: stock prices, physiological signals, weather measurements, and so on. Applying standard machine learning techniques to temporal data requires careful attention to the structure of time, the choice of features, and the evaluation protocol.

4. Objectives and Motivating Examples

4.1 What Do We Want to Predict?

The lectures use financial markets as the primary motivating domain. Two canonical prediction objectives are:

- Whether the price of an asset X days in the future will be **higher or lower** than today.
- Whether a specific share will **outperform its index** over a given horizon.

These objectives generalise to any temporal domain: predicting anomalies in a physiological signal, forecasting energy demand, or anticipating failures in industrial machinery.

4.2 What Do We Have vs. What Do We Need?

We **have**: historical records of observations — for financial data, each row typically contains the date, opening price, high, low, closing price, adjusted close, and volume traded. For heart-rate data, each row contains a timestamp and a BPM value.

We **need**: a table of supervised learning examples where each row contains attribute values (features) derived from a time window ending at time t , and a label or numeric target derived from a future time window starting at $t + 1$.

5. Data in Temporal Series

5.1 Key Parameters

Building a supervised dataset from temporal data requires fixing several parameters:

Parameter	Description
Tick / frequency	The sampling interval: daily, hourly, per second, etc.
Observation window $[t-m, t]$	The historical look-back: features are computed over the m steps ending at the current moment t .
Prediction horizon	Fixed: we predict the value exactly at $t+n$. Variable: we predict any value reached within the interval.
Shift $(+r)$	Each successive training example is generated by sliding the window forward by r steps.

5.2 Fundamental Properties of Temporal Series

- **Autocorrelation:** the statistical similarity between observations as a function of the time lag separating them. High autocorrelation means that knowing the value at time t gives useful information about $t+1$, $t+2$, etc.
- **Stationarity:** the statistical properties of the series (mean and variance) do not change over time. Many classical forecasting methods assume stationarity; non-stationary series must be transformed (e.g., by differencing) before use.
- **Seasonality:** periodic fluctuations that repeat at a known frequency — daily, weekly, yearly, etc. These can be of higher or lower value relative to the long-term trend.

5.3 Output Variable: Future Return

The fixed-horizon future return at time t for a horizon of n steps is:

For a variable horizon (predicting the maximum gain achievable within an interval $[t+n, t+n+d]$):

5.4 Output Variable: Directional Classes

Rather than predicting the exact magnitude of the return, it is often more tractable to predict its direction. The continuous return can be discretised into classes:

The threshold 0.06 (6%) is a design choice that reflects the minimum movement deemed economically significant in a given market. It can be adjusted based on the asset class and trading costs.

6. Features and Indicators in Technical Analysis

6.1 Past Returns as Features

The most direct feature for a financial series is the historical return over the last n steps:

Multiple returns at different look-back windows (e.g., Ret_2 , Ret_5 , Ret_{10} , Ret_{30}) are commonly used as features together, capturing short-, medium-, and long-term momentum.

6.2 Technical Indicator Categories

Technical analysis decomposes price behaviour into five categories:

- **Trend:** long-term directional movements of the price. Trend indicators smooth out short-term fluctuations to reveal the underlying direction.
- **Momentum:** the acceleration (or deceleration) of price changes over a time interval. High positive momentum means prices are rising increasingly fast.
- **Volatility:** the frequency and intensity of price changes in the short term. High volatility implies large, rapid price swings.
- **Volume:** the number of shares (or contracts) traded and its relationship to price. Divergence between price and volume can signal trend reversals.
- **Local extrema (support/resistance):** price levels where the market historically reverses. Local minima (support) and maxima (resistance) are key signals for entry and exit decisions.

6.3 Simple Moving Average (SMA)

The SMA of window size n at time t is:

The SMA lags the price; a longer window produces a smoother but more delayed estimate of the trend. Crossovers between a short-window SMA and a long-window SMA are classic buy/sell signals in technical trading.

6.4 Exponential Moving Average (EMA)

The EMA places more weight on recent observations by using an exponential decay factor α :

The typical choice is $\alpha = 2 / (n + 1)$, where n is the equivalent SMA window size. The EMA reacts faster to recent price changes than the SMA, making it preferred for short-term trading signals.

6.5 Stochastic Oscillator (Momentum Indicator)

The Stochastic Oscillator places the current price within the range of prices over the last n periods:

A value near 100 means the price is close to its recent high (overbought region); a value near 0 means it is close to its recent low (oversold region). Moving averages of the oscillator are used to generate smoothed trading signals.

6.6 Average True Range (ATR — Volatility Indicator)

The True Range (TR) at time t captures the real trading range, accounting for gaps between the previous close and the current high or low:

The ATR is the simple or exponential moving average of $TR(t)$ over a window of n periods. High ATR values indicate high volatility. The Bollinger Bands indicator similarly uses a rolling standard deviation to define upper and lower price envelopes.

6.7 Transforming Indicators into Relative Features

Raw indicator values (e.g., the price level of the SMA) are not directly comparable across different assets or time periods. Two transformations convert them into scale-invariant relative features:

- **Trend of the indicator:** variation of the indicator with respect to its own previous value:
- **Deviation from price:** how far the current price is from the indicator:

6.8 Coding Technical Signals as Features

Instead of using raw indicator values, one can encode established technical signals directly as discrete features. For example, when a short-window SMA crosses above a long-window SMA, a 'Buy' signal is generated; the reverse crossing generates a 'Sell' signal; all other days are coded as 'None'. This categorical feature captures domain expertise about trend reversals in a form directly usable by a classifier.

7. Model Validation in Temporal Series

7.1 Why Standard Cross-Validation Fails

Standard k -fold cross-validation randomly assigns examples to folds, implicitly assuming that observations are exchangeable (independent and identically distributed). In a temporal series this assumption is violated: consecutive observations are correlated. Randomly mixing past and future data into the same fold creates **data leakage** — the model effectively sees future information during training, yielding optimistic and unreliable performance estimates.

Key principle: in temporal validation, the test set must always lie strictly in the future relative to the training set. We evaluate looking forward, just as the model will be deployed.

7.2 Sliding Window Validation

Sliding window validation respects temporal ordering by construction:

- Fix a training window of size W_{train} and a test window of size W_{test} .
- Train the model on the training window, evaluate on the immediately following test window, and record the test error.
- Slide both windows forward by W_{test} steps and repeat.
- The final estimate of generalisation error is the average of the errors recorded across all iterations.

7.3 Accumulative (Expanding-Window) Learning

A variant of the sliding window keeps the start of the training set fixed at the beginning of the series while expanding the training window at each iteration. This approach is appropriate when older data remains relevant and there is no reason to discard it — the model benefits from seeing all available history at each step.

7.4 Day Forward-Chaining Nested Cross-Validation

For hyperparameter tuning in temporal settings, the nested cross-validation scheme extends the sliding window idea to two levels:

- **Outer loop:** multiple train/test splits are generated by forward chaining.
- **Inner loop:** within each outer training window, a further train/validate split (again respecting temporal order) is used for model selection and hyperparameter optimisation.
- The chosen hyperparameters are used to retrain on the full outer training window, and the resulting model is evaluated on the outer test set.
- The final performance metric (e.g., RMSE) is the average of the outer test errors: $(1/K) \sum \text{RMSE}_i$.

7.5 Backtesting

Backtesting goes one step further than measuring classification accuracy: it simulates the actual deployment of the predictive model in a realistic trading (or operational) context. Rather than reporting error rates, backtesting evaluates business-relevant metrics:

- **Profitability / rentability:** the cumulative return achieved by following the model's buy/sell signals.
- **Volatility of returns:** how stable or erratic the equity curve is.
- **Tracking error:** the deviation of the strategy's returns from a benchmark index.
- **Comparison with baselines:** the model's results are compared against simple strategies such as buy-and-hold or a random classifier.

Important caveat: backtesting on historical data is subject to look-ahead bias (using information that would not have been available at decision time) and overfitting to the specific historical period. Results should always be interpreted with caution and validated on out-of-sample data.

Summary: Key Concepts at a Glance

Concept	Core Idea
Feature Selection	Reduce dimensionality by keeping only the most informative attributes.
PCA	Linear projection preserving maximum variance; loses feature semantics.
Sample Error	Fraction of mistakes on a finite dataset — an estimate of true error.
True Error	Probability of misclassification on unseen data from the same distribution.
Confidence Interval	A range that contains the true error with a specified probability.
Paired t-Test	Statistical test to determine whether two learners differ significantly.
Future Return	Percentage price change from t to $t+n$; can be discretised into classes.
SMA / EMA	Moving averages that smooth prices to reveal trend direction.
Stochastic Oscillator	Places current price relative to its recent range; measures momentum.
ATR	Average True Range — measures volatility.
Sliding Window Validation	Temporally ordered train/test splits that respect time causality.
Backtesting	Simulates deployment to evaluate business-relevant performance metrics.